



**Maynooth
University**

National University
of Ireland Maynooth



CS211FZ: Data Structures and Algorithms II

2- Foundations

LECTURER: CHRIS ROADKNIGHT

CHRIS.ROADKNIGHT@MU.IE

Start Here

Order of growth

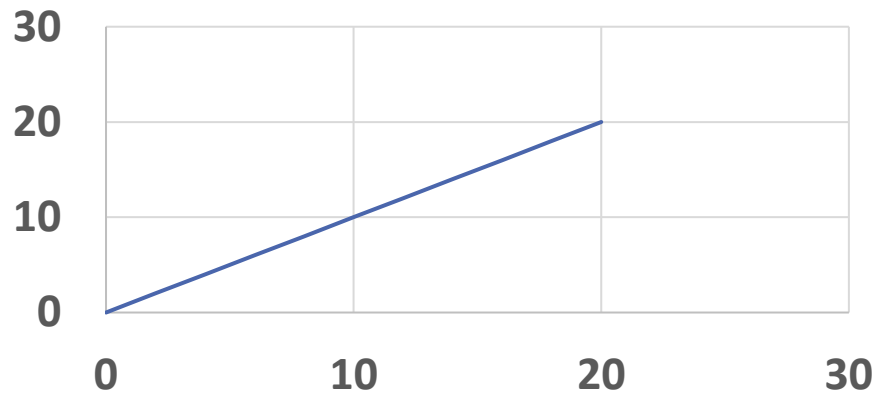
- We now make one more simplifying abstraction: it is the rate of growth, or order of growth, of the running time.
- We consider only the leading terms of a formula, since the lower-order terms are relatively insignificant for large values of n .
- For example, if the rate of growth of an algorithm is $50 + 3x + 100x^2$, we only consider $100x^2$ term.
- We also ignore the leading term's constant coefficient, since constant factors are less significant than the rate of growth in determining computational efficiency for large inputs. Therefore, we select x^2 from $100x^2$ term.
- For linear search, we use $\Theta(n)$ (pronounced “theta of n ”) to denote its running time.

x	$f(x) = x$	$f(x) = 100 + x$	$f(x) = 10x$	$f(x) = x^2$	$f(x) = x^4$	$f(x) = 2^x$	$f(x) = 100 (2^x)$
0	0	100	0	0	0	1	100
2	2	102	20	4	16	4	400
4	4	104	40	16	256	16	1600
6	6	106	60	36	1296	64	6400
8	8	108	80	64	4096	256	25600
10	10	110	100	100	10000	1024	102400
12	12	112	120	144	20736	4096	409600
14	14	114	140	196	38416	16384	1638400
16	16	116	160	256	65536	65536	6553600
18	18	118	180	324	104976	262144	26214400
20	20	120	200	400	160000	1048576	104857600

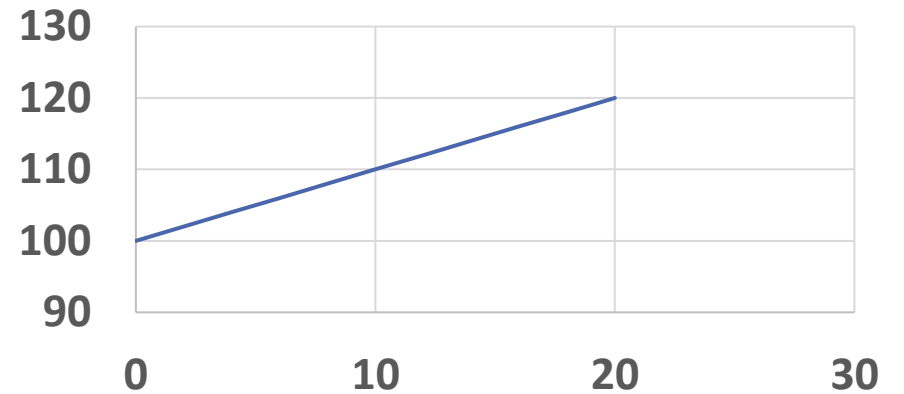
Why only the leading term and without its constant coefficient? (1)

Why only the leading term and without its constant coefficient? (2)

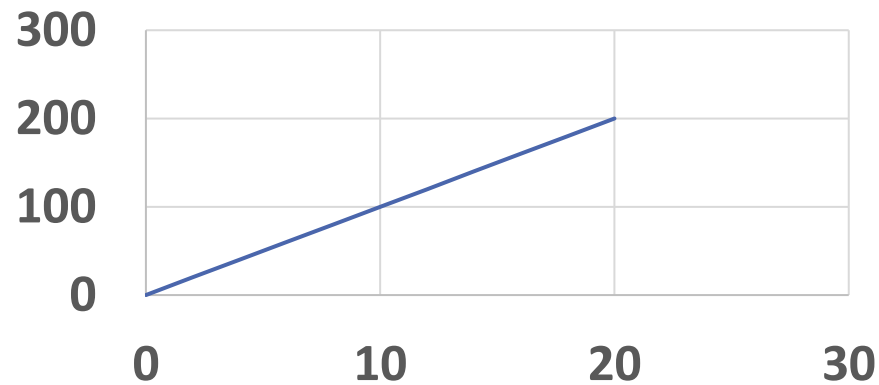
$$f(x)=x$$



$$f(x) = 100+x$$

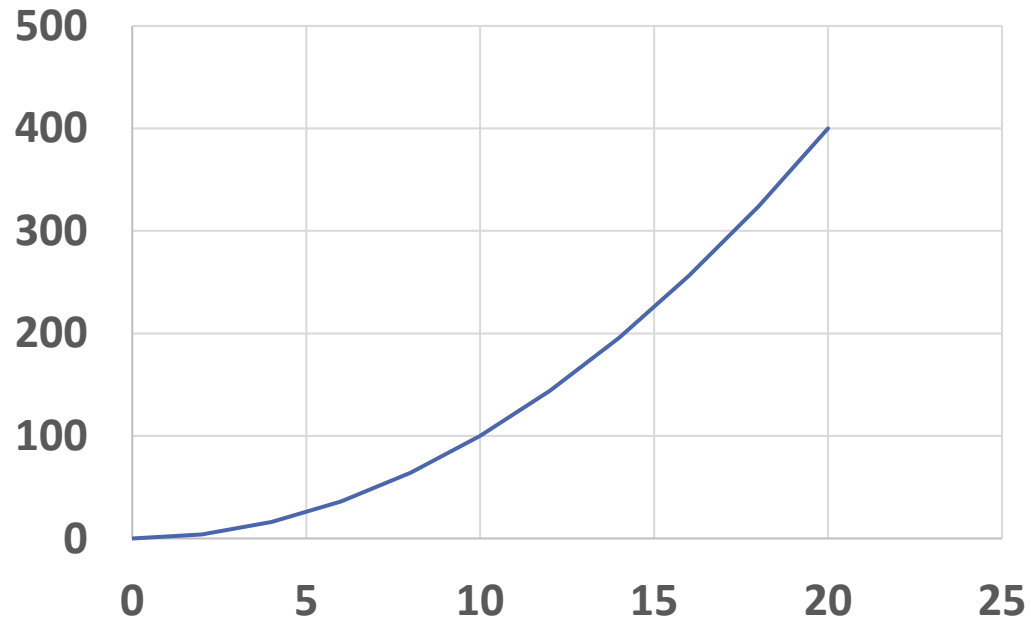


$$f(x) = 10x$$

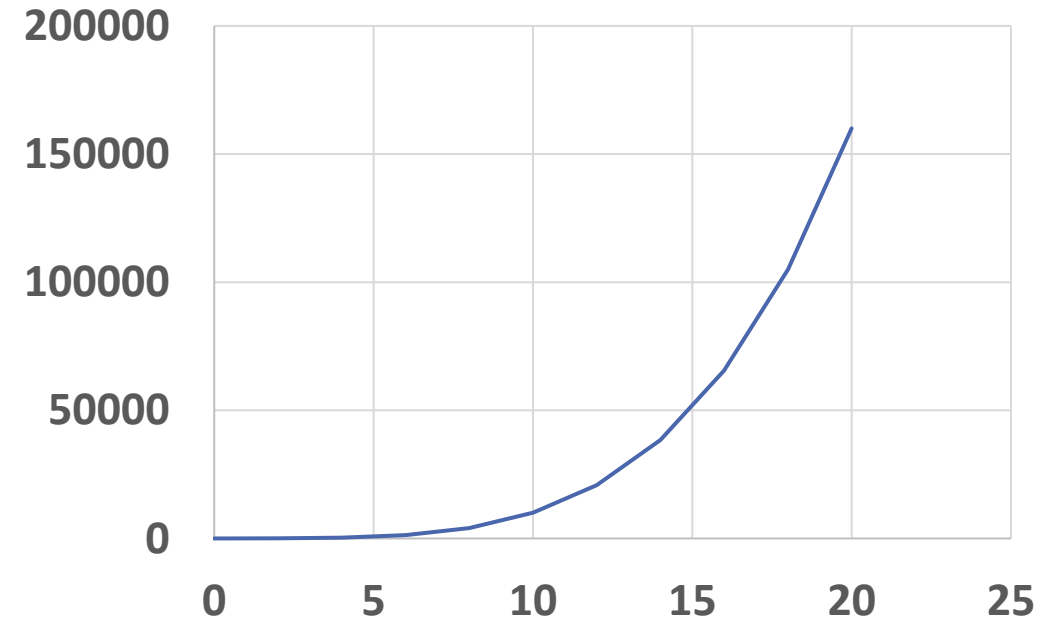


Why only the leading term and without its constant coefficient? (3)

$$f(x) = x^2$$

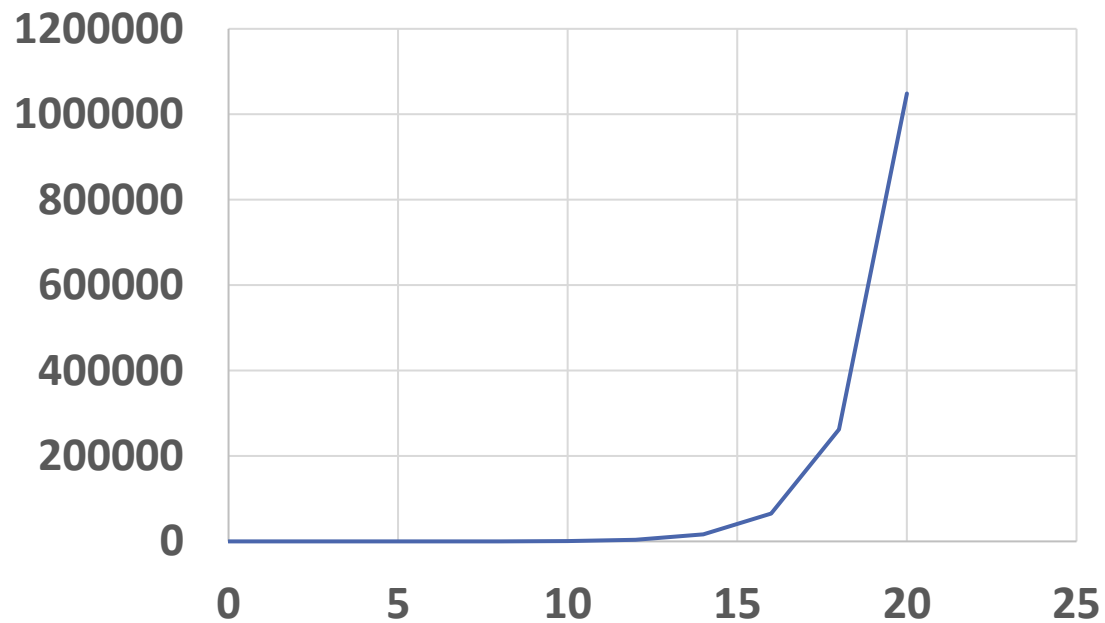


$$f(x) = x^4$$

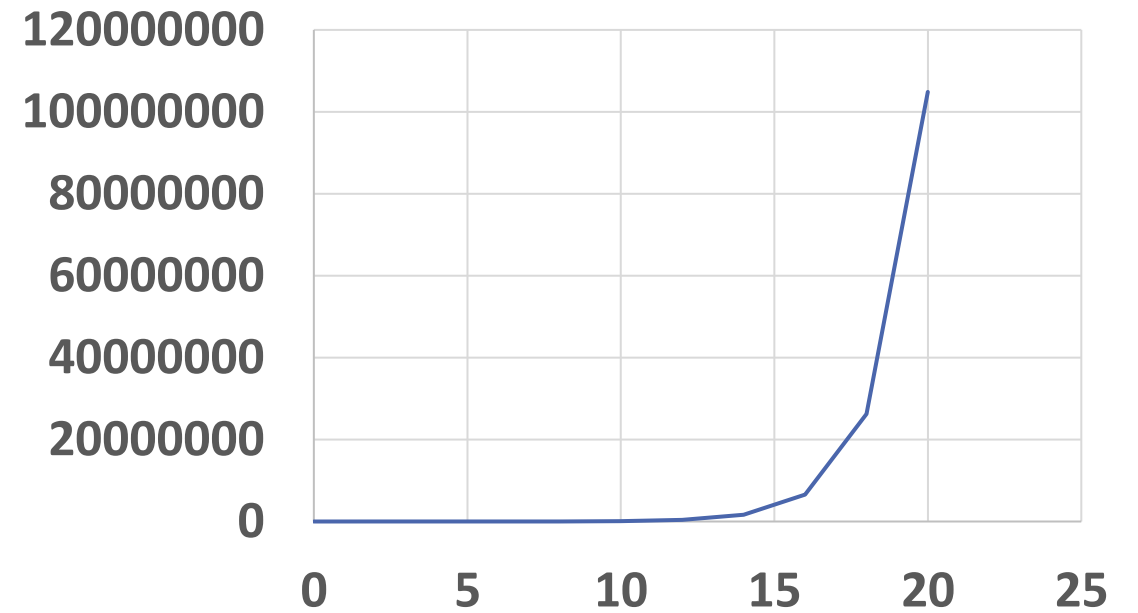


Why only the leading term and without its constant coefficient? (4)

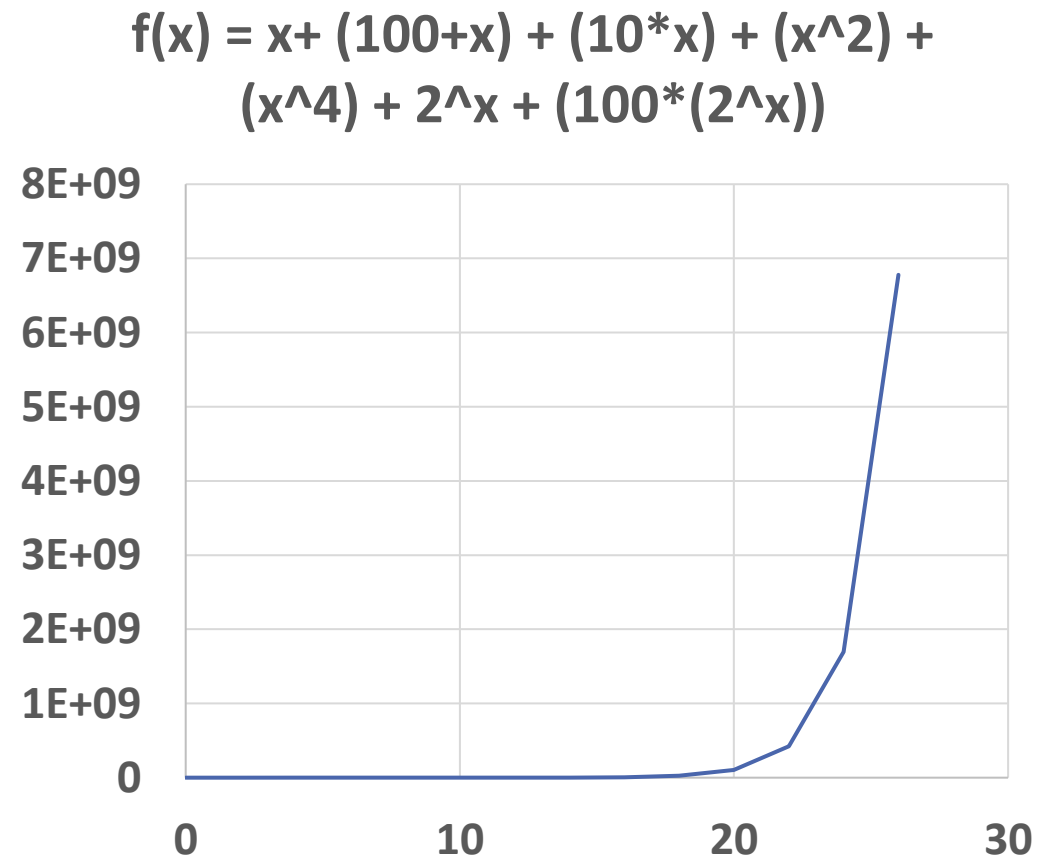
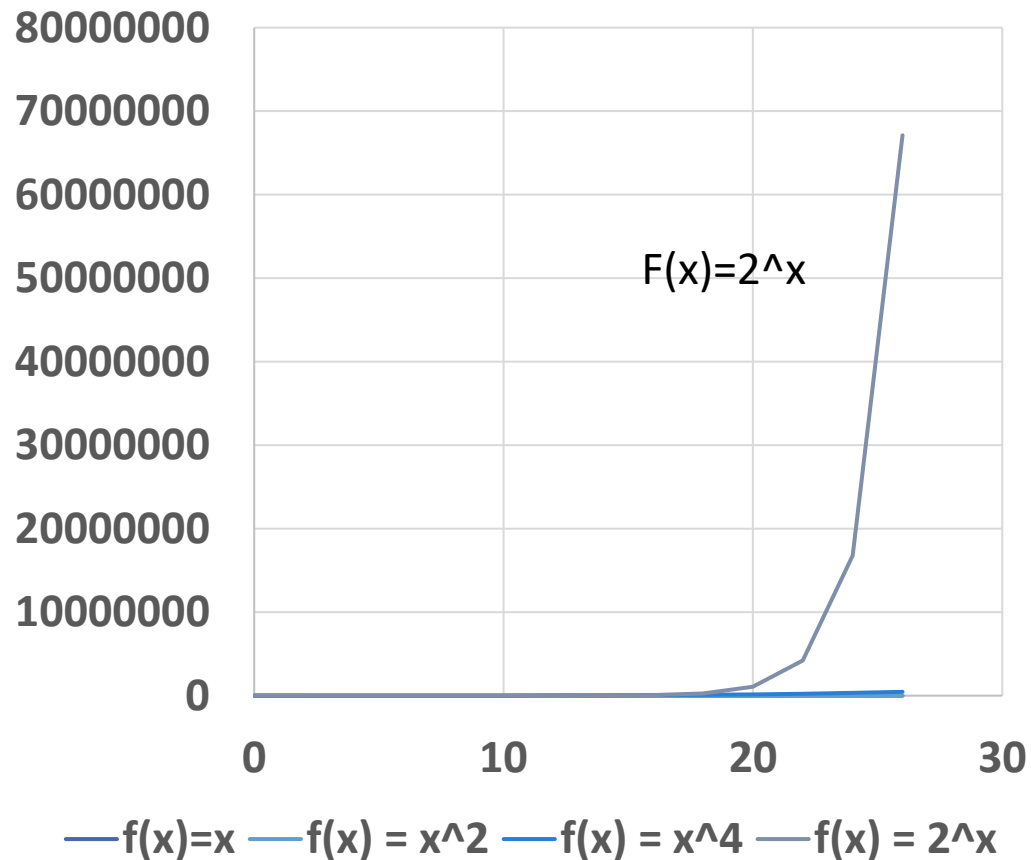
$$f(x) = 2^x$$



$$f(x) = 100 (2^x)$$



Why only the leading term and without its constant coefficient? (5)



Why only the leading term and without its constant coefficient? (6)

- It can be determined that when the number of input (n) becomes bigger, there are significant differences between these terms $100 + x$, $10x$, x^2 , x^4 , 2^x and this term $100(2^x)$.
- In addition, by comparing the graphs of $f(x) = x$, $f(x) = 100 + x$, and $f(x) = 10x$, we can see that constant numbers (100 and 10) did not change the pattern of the rate of growth.
- Similarly, the same conclusion can be extracted from comparing the graphs of $f(x) = x^2$ with $f(x) = 100x^2$.

Why only the leading term and without its constant coefficient? (7)

- The graphs in previous slides illustrate that there is a considerable difference between the leading term (2^x) and the other terms. In addition, it can be determined that there is not any difference between pattern of the leading term and the pattern of the summation of all of the terms.
- Therefore, we consider only the leading terms of a formula without considering the leading term's constant coefficient.
- For example, if the formula is $x + (100 + x) + 10x + x^2 + x^4 + 2^x + 100(2^x)$, the 2^x is dominant so it is only 2^x that interests us in terms of growth.

Memory problem

Memory consumption is another factor, the amount of memory that some of algorithms require may cause memory problem.

Lets look at two algorithms for calculating n^{th} term of Fibonacci sequence.

- 1) Non- Recursive
- 2) Recursive

Fibonacci sequence:

0 1 1 2 3 5 8 13 21 ...

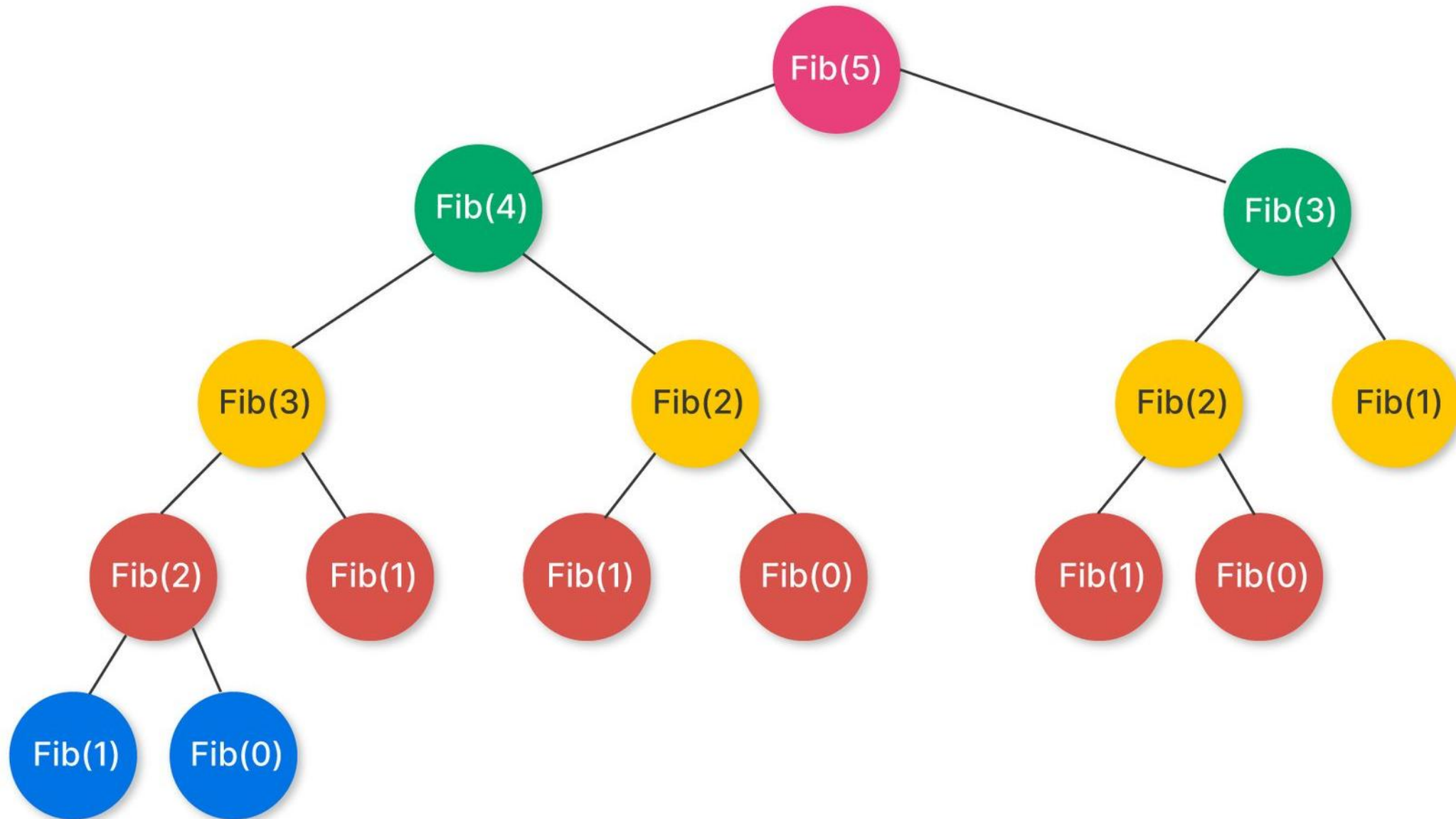
Fibonacci sequence – Non Recursive

```
long Fibo_NonRecursive(int n)
{
    n1 = 0; //declare the first 2 values
    n2 = 1;
    if( n == 0)
        return n1;
    for (i = 2; i <= n; i++) //loop
    {
        sum = n1 + n2; //add new value to sum
        n1 = n2; //move the 2nd number to the 1st place
        n2 = sum;
    }
    return n2;
}
```

Fibonacci sequence – Recursive

```
long Fibo_Recursive(long n)
{
    if (n <= 1)
        return n;
    return Fibo_Recursive(n-1) + Fibo_Recursive(n-2);
}
```

Recursion Graph



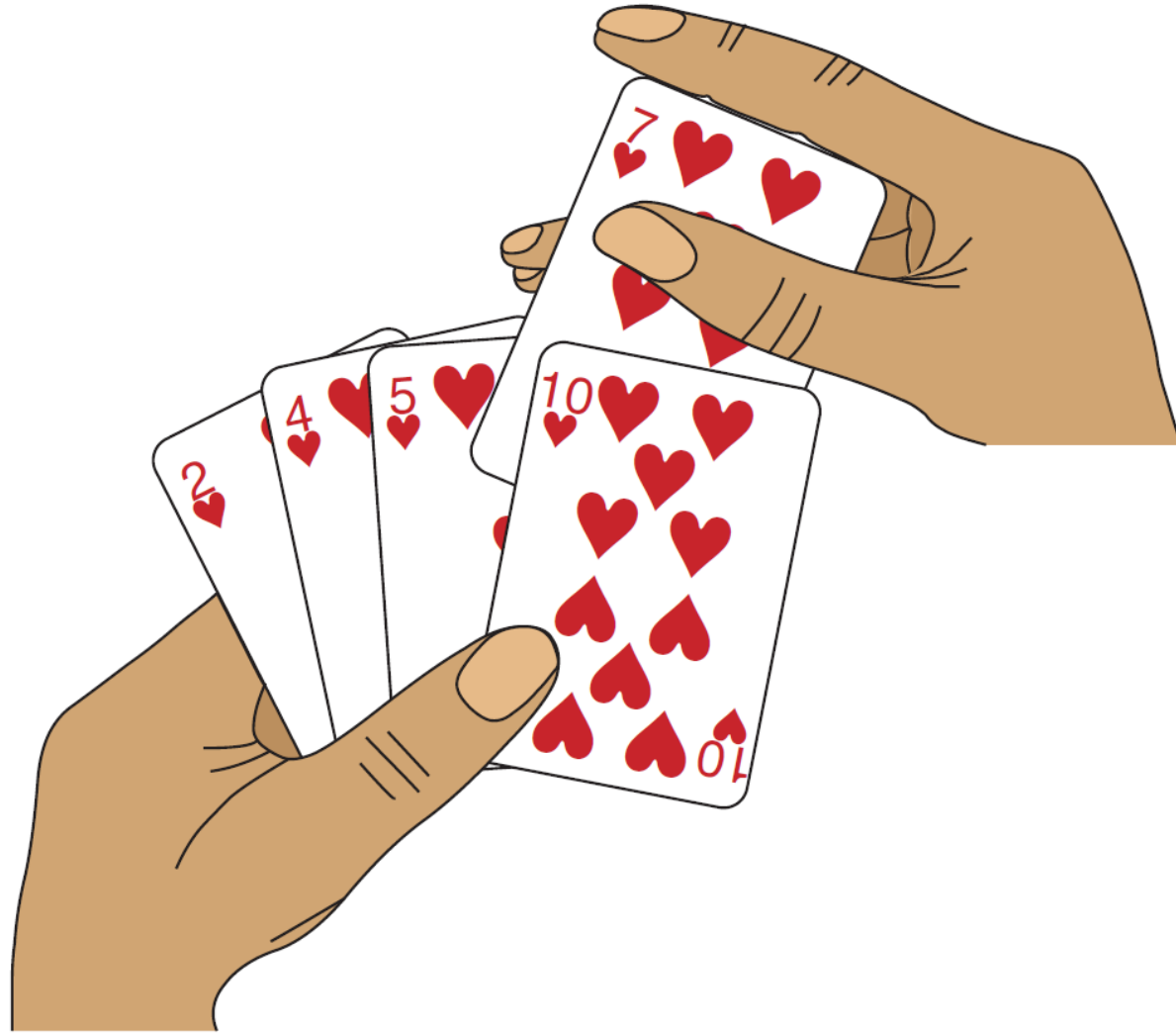
The Fibonacci Algorithms Analysis

We can compare these two algorithms by running their code (shared with you in Moodle).

As can be seen, as the index grows, the recursive algorithm takes significantly more time. In addition, for larger sequences, **stack overflows** will happen.

Stacks are one of the data structures that computer systems use. Therefore, we have seen that memory is one of the problems in some algorithms. Memory usage is important but in this module we are more interested in running time.

In the next sections, we will learn how to compare algorithms formally without running the associated code.



Insertion Sort (2)

Insertion Sort (1)

We want to learn how to formally analyse algorithms by analysing Insertion sort as an example.

Input: A sequence of n numbers $a_1, a_2, \dots, a_n$.

Output: A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

E.g. 3,6,2,1 becomes 1,2,3,6

The numbers that we wish to sort are also known as the **keys**.

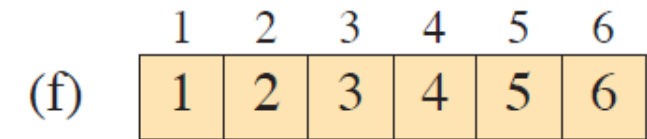
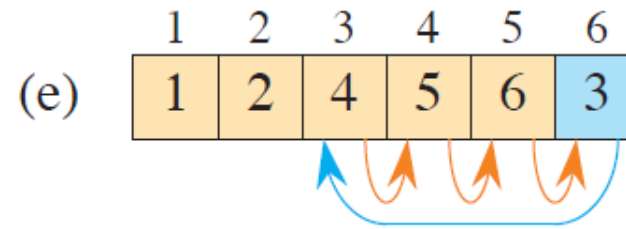
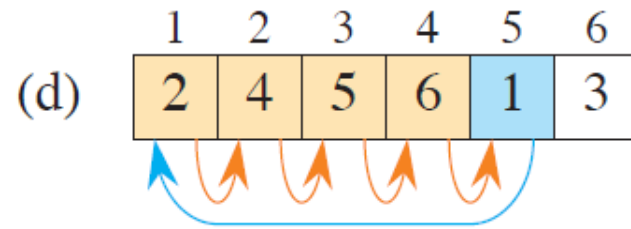
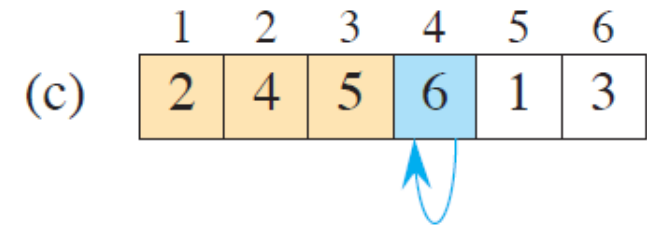
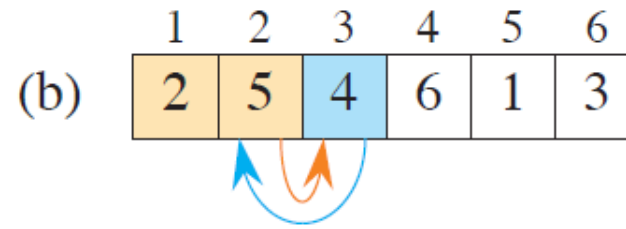
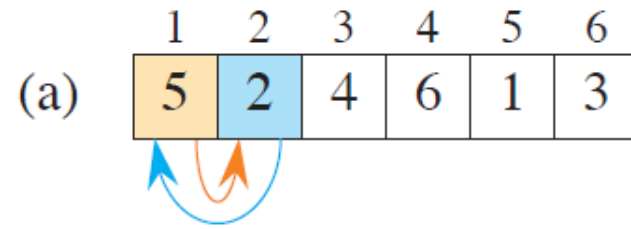
Insertion Sort (3)

Insertion sort is an efficient algorithm for sorting a small number of elements. Insertion sort works the way many people sort a hand of playing cards.

We start with an empty left hand and the cards face down on the table. We then remove one card at a time from the table and insert it into the correct position in the left hand. To find the correct position for a card, we compare it with each of the cards already in the hand, from right to left, as illustrated in previous slide.

At all times, the cards held in the left hand are sorted, and these cards were originally the top cards of the pile on the table.

Insertion Sort (4)



```
INSERTION-SORT(A, n)
1  for i = 2 to n
2      key = A[i]
3      // Insert A[i] into the sorted subarray A[1 : i - 1].
4      j = i - 1
5      while j > 0 and A[j] > key
6          A[j + 1] = A[j]
7          j = j - 1
8      A[j + 1] = key
```

Insertion Sort (5)

```
INSERTION-SORT(A, n) //A is array to be sorted, n is the number of elements
1  for i = 2 to n // Outer Loop, starts at 2 because 1 is already sorted
2      key = A[i] //store current element, key holds the value of the element at index i
           //key is necessary because we might overwrite A[i]
3      // Insert A[i] into the sorted subarray A[1 : i - 1].
4      j = i - 1 //j starts at the index just before i, which is the last element of the
           //sorted subarray (A[1 : i-1]).
5      while j > 0 and A[j] > key //inner loop. A[j] > key (the element at index j is
           //greater than the key, so we need to shift A[j] to make space for key).
6          A[j + 1] = A[j] //we shift the element at index j one position to the right (to
A[j+1]).
           //This creates space for key to be inserted in the correct position.
7          j = j - 1 //After shifting, we move j one position to the left to compare the next
           //element in the sorted subarray with key.
8      A[j + 1] = key
```

What Insertion Sort Does

Insertion Sort works by dividing the array into two parts:

- A sorted portion (initially just the first element).
- An unsorted portion (the remaining elements).

In each iteration, it takes an element from the unsorted portion (starting from the second element, index 2 in this indexing pseudocode) and inserts it into the correct position in the sorted portion, shifting other elements as needed to make space.

Insertion Sort

Quick Facts

Time complexity: $O(n^2)$ in worst/average case (very slow for large arrays).

Space complexity: $O(1)$ — excellent

Stable — equal elements keep their original order.

Best for small arrays or when the array is **almost sorted**.

Loop invariants and the correctness of insertion sort (1)

The index i indicates the “current card” being inserted into the hand. At the beginning of each iteration of the “outer” **for** loop, which is indexed by i , the subarray consisting of elements $A[1 .. i - 1]$ constitute the currently sorted hand, and elements $A[i + 1 .. n]$ correspond to the pile of cards still on the table.

In fact, elements $A[1 .. i - 1]$ are the elements *originally* in positions 1 through $i - 1$, but now in sorted order. We state these properties of $A[1 .. i - 1]$ formally as a **loop invariant**:

“At the start of each iteration of the for loop of lines 1–8, the subarray $A[1 .. i - 1]$ consists of the elements originally in $A[1 .. i - 1]$ but in sorted order.”this is a loop invariant

Loop invariants and the correctness of insertion sort (2)

We use loop invariants to help us understand why an algorithm is correct. We must show three things about a loop invariant:

Initialization: It is true prior to the first iteration of the loop.

Maintenance: If it is true before an iteration of the loop, it remains true before the next iteration.

Termination: When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

Loop invariants and the correctness of insertion sort (3)

We must show that this loop invariant holds prior to the first iteration of the loop, that each iteration of the loop maintains the invariant, that the invariant provides a useful property to show correctness when the loop terminates.

Initialization

We start by showing that the loop invariant holds before the first loop iteration, when $i = 2$. The subarray $A[1 .. i - 1]$, therefore, consists of just the single element $A[1]$, which is in fact the original element in $A[1]$. Moreover, this subarray is sorted (trivially, of course), which shows that the loop invariant holds prior to the first iteration of the loop.

Maintenance

Next, we tackle the second property: showing that each iteration maintains the loop invariant. Informally, the body of the outer for loop works by moving $A[i - 1]$, $A[i - 2]$, $A[i - 3]$, and so on by one position to the right until the proper position for $A[i]$ is found (lines 4–7), at which point the value of $A[i]$ is inserted (line 8).

Termination

Finally, we examine what happens when the loop terminates. For insertion sort, the outer **for** loop ends when i exceeds n , i.e., when $i = n + 1$.

Substituting $n + 1$ for j in the wording of loop invariant, we have that the subarray $A[1 .. n]$ consists of the elements originally in $A[1 .. n]$, but in sorted order. But the subarray $A[1 .. n]$ is the entire array! Hence, the entire array is sorted, which means that the algorithm is correct.

Analysis of insertion sort (1)

INSERTION-SORT(A, n)		<i>cost</i>	<i>times</i>
1	for $i = 2$ to n	c_1	n Or $n-1$
2	$key = A[i]$	c_2	$n - 1$
3	<i>// Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$.</i>	0	$n - 1$
4	$j = i - 1$	c_4	$n - 1$
5	while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6	$A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8	$A[j + 1] = key$	c_8	$n - 1$

Pseudocode

Analysis of insertion sort (2)

Notes:

- $i = 2, 3, \dots, n$, where $n = \text{length}[A]$, we let t_j be the number of times the **while** loop test in line 5 is executed for that value of i .
- When a **for** or **while** loop exits in the usual way (i.e., due to the test in the loop header), the test is executed one time more than the loop body.
- We assume that comments are not executable statements, and so they take no time.

Analysis of insertion sort (3)

The running time of the algorithm is the sum of running times for each statement executed; a statement that takes c_i steps to execute and is executed n times will contribute $c_i n$ to the total running time. To compute $T(n)$, the running time of INSERTION-SORT, we sum the products of the cost and times columns, obtaining

$$\begin{aligned} T(n) = & c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ & + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1) . \end{aligned}$$

Analysis of insertion sort – Best case

The best case occurs if the array is already sorted. For each $i = 2, 3, \dots, n$, we then find that $A[i] \leq key$ in line 5 when i has its initial value of $i - 1$. Thus $t_i = 1$ for $i = 2, 3, \dots, n$, and the best-case running time is

$$\begin{aligned} T(n) &= c_1n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

This running time can be expressed as $an + b$ for *constants* a and b that depend on the statement costs c_k ; it is thus a **linear function** of n .

Analysis of insertion sort – worst case (1)

If the array is in reverse sorted order—that is, in decreasing order—the worst case results. We must compare each element $A[i]$ with each element in the entire sorted subarray $A[1..i-1]$, and so $t_i = i$ for $i = 2, 3, \dots, n$. Noting that

$$\begin{aligned}\sum_{i=2}^n i &= \left(\sum_{i=1}^n i \right) - 1 \\ &= \frac{n(n+1)}{2} - 1\end{aligned}$$

AND

$$\begin{aligned}\sum_{i=2}^n (i-1) &= \sum_{i=1}^{n-1} i \\ &= \frac{n(n-1)}{2}\end{aligned}$$

- For $n = 5$: $2 + 3 + 4 + 5 = 14$, and $\frac{5 \times 6}{2} - 1 = 15 - 1 = 14$.
- For $n = 10$: $2 + 3 + \dots + 10 = 54$, and $\frac{10 \times 11}{2} - 1 = 55 - 1 = 54$.

- For $n=5$: $(2-1) + (3-1) + (4-1) + (5-1) = 1 + 2 + 3 + 4 = 10$, and $\frac{5 \times 4}{2} = 10$.
- For $n=10$: $1 + 2 + \dots + 9 = 45$, and $\frac{10 \times 9}{2} = 45$.

Analysis of insertion sort – worst case (2)

we find that in the worst case, the running time of INSERTION-SORT is

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

This worst-case running time can be expressed as $an^2 + bn + c$ for constants a , b , and c that again depend on the statement costs c_k ; it is thus a quadratic function of n .

Average case??



Growth of Functions

Introduction

Although we can sometimes determine the exact running time of an algorithm, as we did for insertion sort in previous section, the extra precision is not usually worth the effort of computing it.

When we look at input sizes large enough to make only the order of growth of the running time relevant, we are studying the **asymptotic*** efficiency of algorithms.

That is, we are concerned with how the running time of an algorithm increases with the size of the input *in the limit*, as the size of the input increases without bound.

* as the input of a function grows to infinity

Asymptotic Notation

The notations we use to describe the asymptotic running time of an algorithm are defined in terms of functions whose domains are the set of the natural numbers $N = \{0, 1, 2, \dots\}$.

Θ -notation

For a given function $g(n)$, we denote by $\Theta(g(n))$ the set of functions.

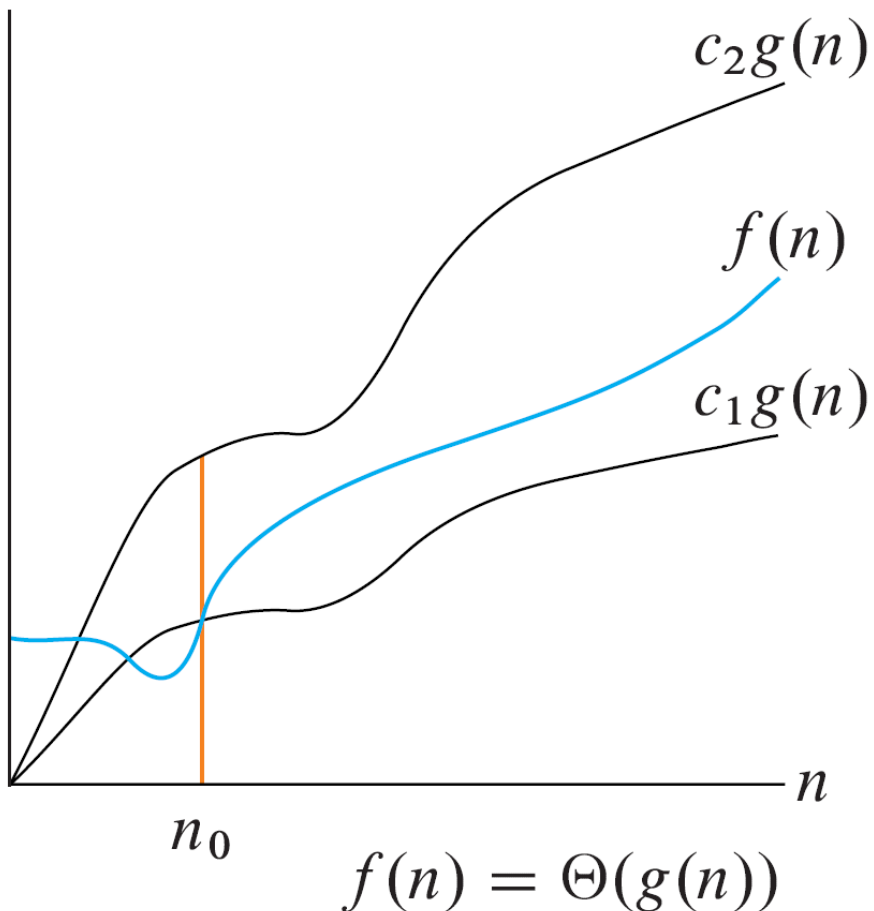
$\Theta(g(n)) = f(n)$: *there exist positive constants c_1 , c_2 , and n_0 such that*

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

We say that $g(n)$ is an asymptotically tight bound for $f(n)$. This definition requires that every member $f(n) \in \Theta(g(n))$ ¹ be non-negative whenever n is sufficiently large.

¹We will use $f(n) \in \Theta(g(n))$ to express the same notion.

Θ -notation



For all values of n to the right of n_0 , the value of $f(n)$ lies at or above $c_1g(n)$ and at or below $c_2g(n)$.

In other words, for all $n \geq n_0$, the function $f(n)$ is equal to $g(n)$ to within a constant factor. We say that $g(n)$ is an **asymptotically tight bound** for $f(n)$.

Example (1)

Show that $\frac{1}{2} n^2 - 3n = \Theta(n^2)$

To do so, we must determine positive constants c_1, c_2 , and n_0 such that

$$c_1 n^2 \leq \frac{1}{2} n^2 - 3n \leq c_2 n^2$$

for all $n \geq n_0$. Dividing by n^2 yields $c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$

The right-hand inequality can be made to hold for any value of $n \geq 1$ by choosing $c_2 \geq 1/2$. Likewise, the left-hand inequality can be made to hold for any value of $n \geq 7$ by choosing $c_1 \leq 1/14$.

Thus, by choosing $c_1 = 1/14$, $c_2 = 1/2$, and $n_0 = 7$, we can verify that $\frac{1}{2} n^2 - 3n = \Theta(n^2)$

Example (2)

We can also use the formal definition to verify that $6n^3 \neq \Theta(n^2)$. Suppose for the purpose of contradiction that c_2 and n_0 exist such that $6n^3 \leq c_2 n^2$ for all $n \geq n_0$.

But then $n \leq c_2/6$, which cannot possibly hold for arbitrarily large n , since c_2 is constant.

Lower-order terms

Intuitively, the lower-order terms of an asymptotically positive function can be ignored in determining asymptotically tight bounds because they are insignificant for large n . When n is large, even a tiny fraction of the highest-order term suffices to dominate the lower-order terms.

Thus, setting c_1 to value that is slightly smaller than the coefficient of the highest-order term and setting c_2 to value that is slightly larger permits the inequalities in the definition of Θ -notation to be satisfied.

The coefficient of the highest-order term can likewise be ignored, since it only changes c_1 and c_2 by a constant factor equal to the coefficient.

O-notation (1)

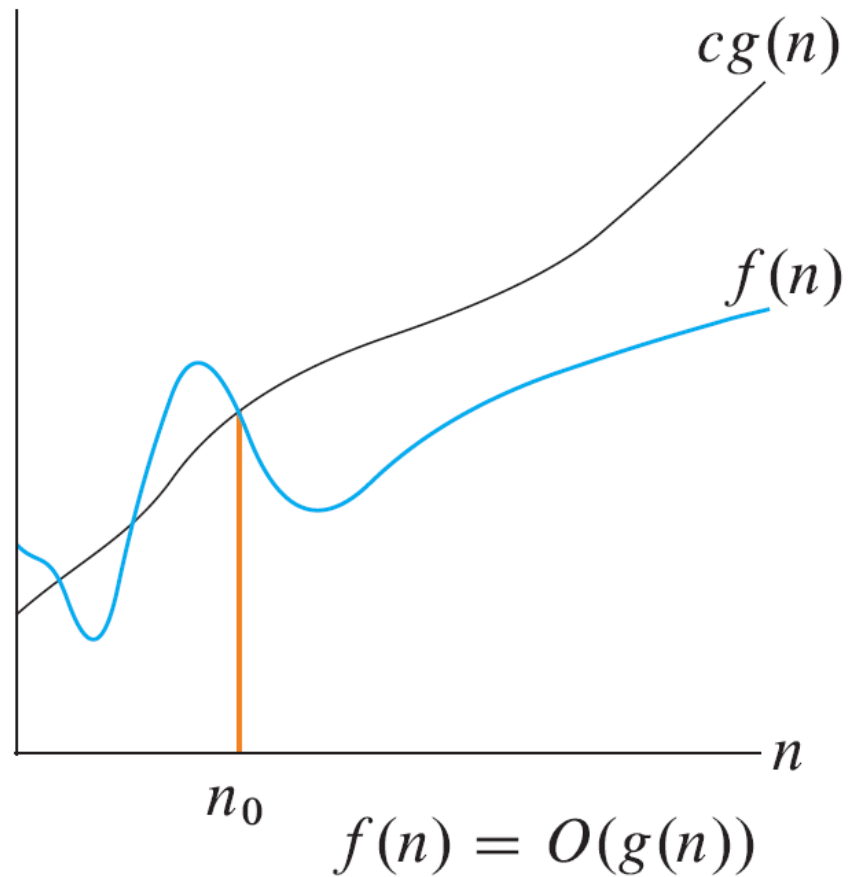
The Θ -notation asymptotically bounds a function from above and below. When we have only an **asymptotic upper bound**, we use O-notation. For a given function $g(n)$, we denote by $O(g(n))$ (pronounced “big-oh of g of n ” or sometimes just “oh of g of n ”) the set of functions

$$\Theta(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that} \\ 0 \leq f(n) \leq c g(n) \text{ for all } n \geq n_0\}$$

We use O-notation to give an upper bound on a function, to within a constant factor. We write $f(n) = O(g(n))$ to indicate that a function $f(n)$ is a member of the set $O(g(n))$.

Note that $f(n) = \Theta(g(n))$ implies $f(n) = O(g(n))$.

O-notation (2)



For all values n to the right of n_0 , the value of the function $f(n)$ is on or below $g(n)$.

O-notation (3)

Using O-notation, we can often describe the running time of an algorithm merely by inspecting the algorithm's overall structure. For example, the doubly nested loop structure of the insertion sort algorithm immediately yields $O(n^2)$ upper bound on the worst-case running time.

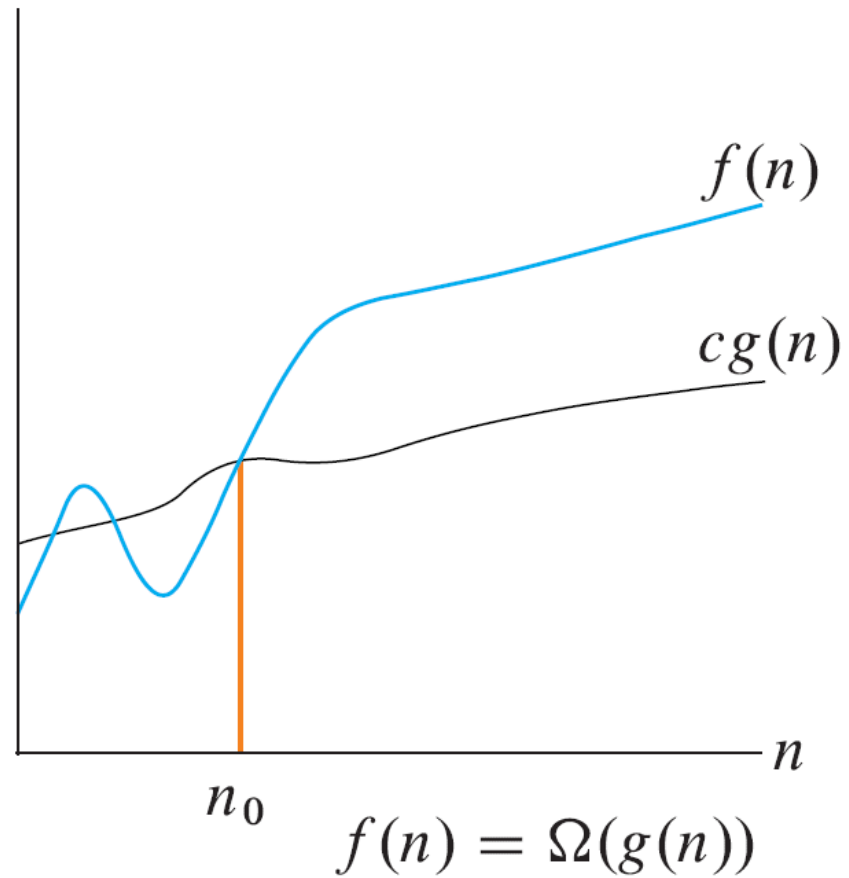
Technically, it is an abuse to say that the running time of insertion sort is $O(n^2)$, since for a given n , the actual running time varies, depending on the particular input of size n . When we say “the running time is $O(n^2)$ ”, we mean that there is a function $f(n)$ that is $O(n^2)$. Equivalently, we mean that **the worst-case running time** is $O(n^2)$.

Ω -notation (1)

Just as O -notation provides an asymptotic *upper* bound on a function, Ω -notation provides an ***asymptotic lower bound***. For a given function $g(n)$, we denote by $\Omega(g(n))$ (pronounced “big-omega of g of n ” or sometimes just “omega of g of n ”) the set of functions

$$\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that}$$
$$0 \leq c g(n) \leq f(n) \text{ for all } n \geq n_0\}$$

Ω -notation (2)



For all values n to the right of n_0 , the value of $f(n)$ is on or above $cg(n)$.

Ω -notation (3)

The running time of insertion sort therefore falls between (n) and $O(n^2)$, since it falls anywhere between a linear function of n and a quadratic function of n .

When we say that the running time of an algorithm is $\Omega(g(n))$, we mean that no matter what particular input of size n is chosen for each value of n , the running time on that input is at least a constant times $g(n)$, for sufficiently large n .

Equivalently, we are giving a lower bound on the best-case running time of an algorithm.

o-notation (1)

The asymptotic upper bound provided by O -notation may or may not be asymptotically tight. The bound $2n^2 = O(n^2)$ is asymptotically tight, but the bound $2n = O(n^2)$ is not. We use o -notation to denote an upper bound that is **not** asymptotically tight. We formally define $o(g(n))$ (“little-oh of g of n ”) as the set

$$o(g(n)) = \{f(n): \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < c g(n) \text{ for all } n \geq n_0\}$$

For example, $2n = o(n^2)$, but $2n^2 \neq o(n^2)$

ω -notation

By analogy, ω -notation is to Ω -notation as o -notation is to O -notation. We use ω -notation to denote a lower bound that is not asymptotically tight. One way to define it is by $f(n) \in \omega(g(n))$ if and only if $g(n) \in o(f(n))$.

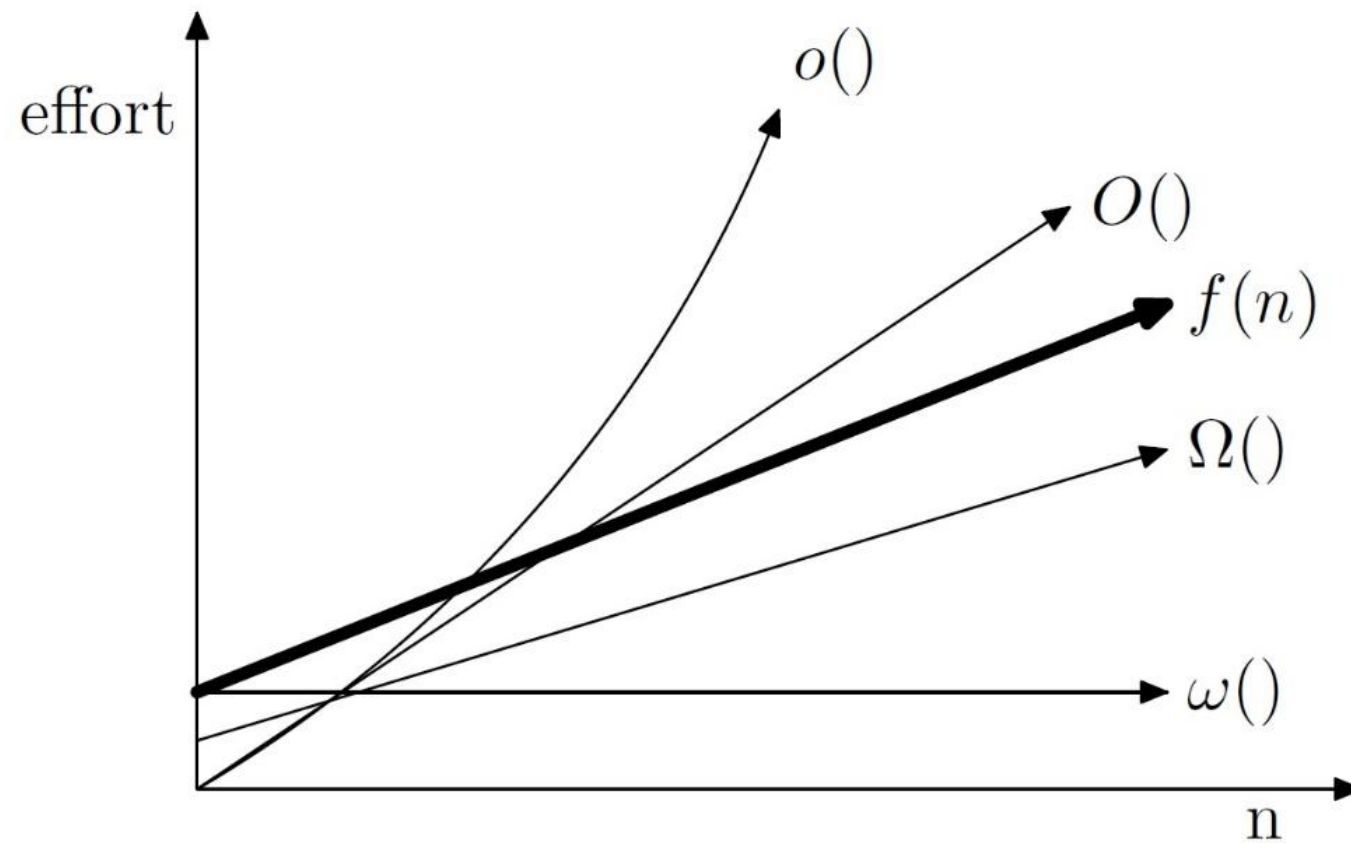
Formally, we define $\omega(g(n))$ (“little-omega of g of n”) as the set

$$\omega(g(n)) = \{f(n) : \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq c g(n) < f(n) \text{ for all } n \geq n_0\}$$

For example, $\frac{n^2}{2} \in \omega(n)$, but $\frac{n^2}{2} \notin \omega(n^2)$

The relation $f(n) \in \omega(g(n))$ implies that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$

Comparing notations



Summary

An easy way to remember the these notations:

$f(n) = O(g(n))$ is like $a \leq b$,

$f(n) = \Omega(g(n))$ is like $a \geq b$,

$f(n) = \Theta(g(n))$ is like $a = b$,

$f(n) = o(g(n))$ is like $a < b$,

$f(n) = \omega(g(n))$ is like $a > b$.